



Звіт про оригінальність

● Оцінка схожості

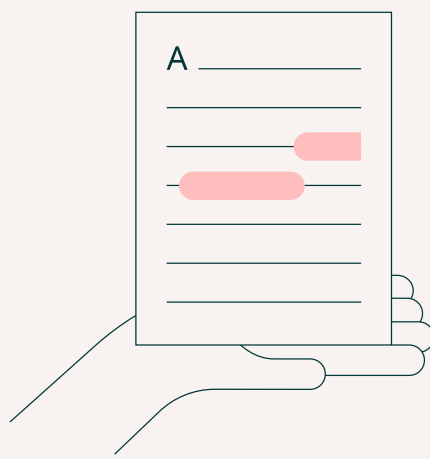
% 5

● Ризик плагіату

СЕРЕДНІЙ

👤 Offiks ⌚ 2026-06-20 08:30

Посилання на звіт: 1bgVy / Посилання користувача: sjWr



Ось вона – Ваша звіт про оригінальність!

Ми раді повідомити, що перевірка вашого документа завершена, і результати вже готові! Наші алгоритми старанно працювали, щоб знайти збіги в наших базах даних.

На наступних сторінках ви знайдете результати перевірки:

Бали

Збіги

Посилання

Ваш документ було перевірено за такими джерелами:

- ☒ База даних інтернет-джерел
- ☐ База даних наукових статей
- ☐ Глибока перевірка (наш вдосконалений алгоритм)

Бали



Збіги тексту	5%
Перефразування	0%
Цитований текст	0%
Неправильне цитування	0%
Збігів не знайдено	95%

Ризик плагіату

СЕРЕДНІЙ

Ризик плагіату вказує, як збіги тексту розподілені по документу. Вищий ризик виникає, коли збіги з'являються близько один до одного, наприклад, у тому самому абзаці або розділі.

Оцінка схожості

Оцінка схожості показує, скільки слів або символів у вашому документі збігаються з текстами інших документів, включаючи перефразовані тексти або неправильні цитати.



5

Збіги

11 МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ **1** ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА **2** Факультет математики, фізики та інформаційних
технологій Кафедра математичного забезпечення комп'ютерних систем КУРСОВИЙ
ПРОЄКТ з освітнього компоненту «Програмування» на тему: СТВОРЕННЯ ІЄРАРХІЇ
КЛАСІВ НА ТЕМУ «Спрощений варіант №10» Студентки 1 курсу, денна форма навчання
спеціальність F7 «Комп'ютерна інженерія» Манчук Юлія Олександрівна Керівник: к.т.н.,
доц. Антоненко О.С. **10** Національна шкала: _____ Кількість
балів: _____ ECTS: _____ Дата захисту: _____
Члени комісії _____ Антоненко О.С. **1** (підпис) (прізвище та ініціали) _____ **1** Шестопалов
С.В. **1** (підпис) (прізвище та ініціали) _____ **1** _____ **1** (підпис) (прізвище та
ініціали) ОДЕСА - 2026

12 2 ЗМІСТ стор. ВСТУП

.....	3 ПОСТАНОВКА
ЗАВДАННЯ	4 1. ТЕОРЕТИЧНА ЧАСТИНА
.....	6 1.1 Поняття ООП
.....	6 1.2 Опис об'єктної частини
.....	7 2. ПРАКТИЧНА ЧАСТИНА
.....	9 2.1 Опис класів
.....	9 2.3 Інтерфейс
.....	11 ВИСНОВКИ
.....	14 СПИСОК ЛІТЕРАТУРИ
.....	15

3 ВСТУП Мета роботи – створити ієрархію класів за спрощеним варіантом №10 та її
застосуванні для вирішення прикладної задачі, визначеної у варіанті завдання. У **9** ході
виконання роботи необхідно реалізувати концепції об'єктно-орієнтованого
програмування: інкапсуляцію, успадкування та поліморфізм. Згідно з вимогами
завданням, слід розробити систему взаємопов'язаних класів, яка описуватиме структуру
і функціонування продуктового магазину. Для цього потрібно розробити базовий клас
Person та його спадкоємців, створити конструктори, закриті поля та методи для роботи
з об'єктами. У програмі слід реалізувати гнучкий сценарій роботи через текстове меню

для додавання, видалення та виведення інформації про об'єкти.

4 ПОСТАНОВКА ЗАВДАННЯ Спрощений варіант 10. 1) Взяти клас Person із четвертої лабораторної (див. таблицю 1 Розробити клас Person, який описує людину, з наступними можливостями: а) Поля: ПІБ fullName; дата народження day, month, year; зріст height; вага weight; стать gender; номер телефона phoneNumber. б) Методи: перевірити чи має зріст понад 200 см? якщо номер телефону починається з 0, додати на початку +38. с) Створити конструктор без параметрів та конструктор з параметрами. d) Створити метод «змінити номер телефону». е) При створенні людини (у конструкторі) виконувати форматування номера телефону. f) Реалізувати метод виведення інформації g) Реалізувати два спадкоємці класу Person (див. таблицю 2 за варіантами). У спадкоємцях перевизначити метод print(). 2) Студент: додаткові поля: спеціальність, курс, група. 3) Робітник: додаткові поля: компанія, посада, зарплата.

5 4) Створити програмну систему з можливістю додавання, видалення, виведення інформації про екземпляри типу Person та його спадкоємців (додатково: пошук). Використовувати вектор вказівників на Person.

6 1.ТЕОРЕТИЧНА ЧАСТИНА 1.1 Поняття ООП Об'єктно-орієнтоване програмування – це концепсія, в межах якої програма розглядається як сукупність об'єктів, що обмінюються повідомленнями [1]. Кожен об'єкт є екземпляром певного класу, а класи, своєю чергою, утворюють ієрархію на основі наслідування. Розробник спочатку визначає класи (шаблони), а під час виконання програми на їхній основі створюються конкретні об'єкти. Нові класи можуть успадковувати властивості від базових, що й породжує ієрархічну структуру. Будь-який стиль програмування має свою концептуальну базу [2]. Для об'єктно-орієнтованого стилю такою базою є об'єктна модель, яка ґрунтується на трьох основних принципах [3]: а) інкапсуляція; б) поліморфізм; с) наслідування. Інкапсуляція – механізм, який поєднує в одному об'єкті дані і методів їхньої обробки, а також приховування внутрішньої реалізації від зовнішнього світу, що запобігає помилковому чи несанкціонованому втручанням [2]. Наслідування – процес, завдяки якому один об'єкт (або клас) може набути властивостей іншого, доповнюючи їх власними, унікальними характеристиками. Це дозволяє будувати ієрархії класів від загального до конкретного [2]. Поліморфізм – здатність одного й того самого імені (наприклад, назви методу) позначати різні дії залежно від типу об'єкта, якому адресоване

7 повідомлення [2]. 6 На практиці це означає, що об'єкт сам обирає відповідний внутрішній метод відповідно до отриманих даних. Головна мета поліморфізму – забезпечити єдиний інтерфейс для класу споріднених операцій. 1.2 Опис об'єктної частини Програма інформаційну систему обліку та керування даними, яка побудована на базі об'єктно-орієнтованої ієрархії із застосуванням принципів успадкування,

інкапсуляції та поліморфізму. Основою архітектури є базовий клас `Person`, який описує людину та містить її загальні анкетні характеристики: ПІБ, дата народження, зріст, вагу, стать і номер телефону. При створенні об'єкта автоматично виконується перевірка та форматування номера телефону: якщо він має 10 цифр і починається з «0», додається префікс «+38». Метод `IsTall()` дозволяє визначити, чи перевищує зріст людини 200 см. Для забезпечення динамічного керування та поліморфної поведінки всі екземпляри базового та похідних класів зберігаються в єдиній колекції – вектори вказівників на `Person` (`vector<Person*>`). Користувач системи взаємодіє з базовим класом `Person` та двома його прямими спадкоємцями: а) клас `Student` (Студент) додатково зберігає спеціальність, курс навчання та назву групи; б) клас `Employee` (Робітник) містить дані про компанію, посаду та зарплату. Усі три класи мають метод `print()`, що дозволяє виводити повну інформацію про об'єкт відповідно до його фактичного типу. Функції `prinAll()`, `addStudent()`, `addEmployee()`, `searchPerson()`, `deletePerson()` та `cleanup()` у файлі `main.cpp` реалізують сценарій роботи через

8 текстове меню. Користувач може у довільному порядку переглядати записи, додавати нових студентів або працівників, шукати за ПІБ та видаляти записи за номером у списку. При завершенні роботи програми функція `cleanup()` звільняє динамічно віділенню пам'ять для всіх об'єктів.

9 2.ПРАКТИЧНА ЧАСТИНА 2.1 Опис класів Діаграму класів наведено в додатку 1. Розглянемо більш детально структуру кожного класу. 1) Базовий клас «`Person`»: а) Приватні поля: `fullName`, `day`, `month`, `year`, `height`, `weight`, `gender`, `phoneNumber` б) Конструктори: □ Конструктор без параметрів; □ Конструктор з параметрами. с) Методи: □ `formatPhone()` – внутрішній метод автоматичного виправлення формату номера: якщо телефон має 10 символів і починається з «0», додає «+38»; інакше, якщо формат некоректний, встановлює «`Nekorektniy nomer`»; □ `isTall()` – повертає `true`, якщо зріст особи перевищує 200 см; □ `changePhone()` – змінює номер телефону з повторним форматуванням; □ `getName()`, `getDay()`, `getMonth()`, `getYear()`, `getHeight()`, `getWeight()`, `getGender()`, `getPhone()` – методи доступу до полів; □ `print()` – віртуальний метод виведення базової інформації про людину, включно з результатом перевірки зросту; □ віртуальний деструктор `~Person()`. 2) Клас «`Student`», нащадок класу «`Person`»: а) Додаткові поля: `shelfLife`, `amountCalories`. б) Конструктори:

10 □ Конструктор без параметрів; □ Конструктор з параметрами. с) Методи: □ `print()` – перевизначений метод, який виводить позначку `[STUDENT]`, ПІБ, стать, дату народження, телефон, спеціальність, курс і групу. 3) Клас «`Employee`», нащадок класу «`Person`»: а) Додаткові поля: `company`, `position`, `salary` б) Конструктори: □ Конструктор без параметрів; □ Конструктор з параметрами. с) Методи: □ `print()` – перевизначений метод, який виводить позначку `[ROBITNYK]`, ПІБ, стать, дату народження, зріст, вагу, телефон, компанію, посаду та зарплату. 4) Головна програма (`main.cpp`): а) Глобальна колекція:

vector<Person*> people b) Функції: □ printAll() – виводить усі записи з використанням поліморфного виклику print(); □ addStudent() – інтерактивне додавання студента з перевіркою коректності дати народження, зросту та ваги; □ addEmployee() – інтерактивне додавання працівника з аналогічною валідацією; □ searchPerson() – пошук записів за підрядком у полі ПІБ; □ deletePerson() – видалення запису за номером у списку зі звільненням пам'яті;

11 □ cleanup() – очищення колекції при завершенні програми. c) Початкові тестові дані: при запуску програми автоматично додаються три записи – два студенти та один працівник, серед яких є людина зі зростом понад 200 см (Veleten Maksym Olehovych, 205 см). 2.2 Інтерфейс Під час запуску програми користувач бачить головне вікно з текстовим меню (Рис. 2.1). Рисунок 2.1 – Головне меню програми. Користувачеві пропонується ввести цифру від 0 до 5 для вибору необхідної операції. Якщо обрати пункт 1, програма виводить на екран повний список усіх людей, зареєстрованих у системі (Рис. 2.2). Рисунок 2.2 – Виведення списку всіх людей.

12 Користувач може скористатися пунктом 2. Яке відповідає за додавання студента, виводяться його дані: ПІБ, дату народження, зріст, вага, стать, номер телефона, спеціальність, курс, група (Рис 2.3). Рисунок 2.3 – Вікно додавання студента. Аналогічно пункту 3 користувач додає працівника та вводить його дані: ПІБ, дату народження, зріст, вага, стать, номер телефона, компанія, посада, зарплата (Рис 2.4). Рисунок 2.4 – Вікно додавання робітника.

13 Після вибору пункту 4 програма шукає людину за ПІБ (Рис 2.5). Рисунок 2.5 – Вікно пошуку людини за ПІБ. Після вибору пункту 5 програма просить ввести номер зі списку який хочете видалити (Рис 2.6). Рисунок 2.6 – Вікно видалення людини за номером списку. Після вибору пункту 0 на екран виводиться прощальне повідомлення, і робота програми завершується (Рис 2.7). Рисунок 2.7 – Вікно завершення програми

14 ВИСНОВКИ Програма пройшла тестування на початкових та введених користувачем даних, продемонструвавши стабільну роботу та коректне звільнення пам'яті при завершенні. Результати роботи підтвердили ефективність використання об'єктно-орієнтованих технологій для створення прикладного програмного забезпечення. Під час виконання курсової роботи я розробив програму на мові C++. У ході роботи було реалізовано ієрархію класів, де базовий клас Person містить загальні антропометричні та контактні дані (ПІБ, дата народження, зріст, вага, стать, телефон), а похідні класи Student та Employee розширюють цей функціонал специфічними властивостями (спеціальність, курс, група; компанія, посада, зарплата). Особливу увагу приділено принципу інкапсуляції: доступ до полів класу здійснюється через методи-геттери, а форматування телефону виконується внутрішнім методом formatPhone(). Реалізовано перевірку зросту понад 200 см методом isTall(), що відповідає вимогам варіанту № 10.

Для управління базою даних використано динамічну колекцію `vector<Person*>`, яка завдяки механізму поліморфізму **13** дозволяє зберігати об'єкти різних типів в одному списку та коректно опрацьовувати їх за допомогою віртуального методу `print()`. Розроблений інтерфейс у вигляді текстового меню забезпечує зручну взаємодію з користувачем, дозволяючи виконувати такі операції: а) додавання нових записів (студентів і працівників); б) перегляд повної бази даних; с) видалення записів за номером списку;

15 д) пошук інформації за ПІБ; е) автоматичне форматування номера телефону та перевірка зросту. Програма пройшла тестування на початкових та введених користувачем даних, продемонструвавши стабільну роботу та коректне звільнення пам'яті при завершенні. Результати роботи підтвердили ефективність використання об'єктно-орієнтованих технологій для створення прикладного програмного забезпечення.

СПИСОК ЛІТЕРАТУРИ 1. С. А. Ярошко, Основи об'єктно-орієнтованого

3 програмування з ілюстраціями на Borland Pascal та Borland Pascal for Windows

[Електронне видання] / Львів 1998. – Режим доступу:

<https://ami.lnu.edu.ua/wp-content/uploads/2020/04/OOP.pdf> 2. Web Library, Основні поняття ООП. [Електронне видання] / Тернопіль 2026. – Режим доступу:

<http://weblib.com.ua/blog/osnovni-ponyattya-oop> 3. W3School, C++ OOP (Object-Oriented Programming). Режим доступу: https://www.w3schools.com/cpp/cpp_oop.asp 4.

Репозиторій GitHub на курсовий – Режим доступу:

<https://github.com/yuliamanchuk-creator/Kurosoyaya1>

16 Додаток А. Діаграма класів Рисунок А1. – Діаграма класів

17 Додаток Б. Код класу `#include "Person.h" #include <iostream> 14 using namespace std; Person::Person() : fullName("Невідомо"), day(1), month(1), year(2000), height(0.0), weight(0.0), gender("невідомо"), phoneNumber("") { } Person::Person(const string& fullName, int day, int month, int year, double height, double weight, const string& gender, const string& phoneNumber) : fullName(fullName), day(day), month(month), year(year), height(height), weight(weight), gender(gender), phoneNumber(phoneNumber) { formatPhone(); } void Person::formatPhone() { if (phoneNumber.length() == 10 && phoneNumber[0] == '0') { phoneNumber = "+38" + phoneNumber; } else if (phoneNumber.length() != 13 || phoneNumber[0] != '+') { phoneNumber = "Некоректний номер"; } } void Person::changePhone(const string& newPhone) { phoneNumber = newPhone; formatPhone(); } bool Person::isTall() const { return height > 200; } string Person::getName() const { return fullName; } int Person::getDay() const { return day; } int Person::getMonth() const { return month; } int Person::getYear() const { return year; } double Person::getHeight() const { return height; } double Person::getWeight() const { return weight; } string Person::getGender() const { return gender; } string Person::getPhone() const { return phoneNumber; } void Person::print() const { cout << fullName << " | " << day << " " << month`


```
<< "." << year << " | Zrist: " << height << " cm | Vaga: " << weight << " kg" << " | Stat: " <<
gender << " | Tel: " << phoneNumber << " | Zrist > 200: " << (isTall() ? "Tak" : "Ni"); }
Person::~~Person() Лістинг Б.1 – Клас Person
```

```
18 #include "Student.h" #include <iostream> 5 using namespace std; Student::Student() :
Person(), specialty("KI"), course(1), groupName("KI-26") { } Student::Student(const string&
fullName, int day, int month, int year, double height, double weight, const string& gender,
const string& phoneNumber, const string& specialty, int course, const string& groupName) :
Person(fullName, day, month, year, height, weight, gender, phoneNumber),
specialty(specialty), course(course), groupName(groupName) { } void Student::print() const {
cout << "[STUDENT] " << getName() << " (Stat: " << getGender() << ")" << " | Data
narodzhennya: " << getDay() << "." << getMonth() << "." << getYear() << " | Tel: " <<
getPhone() << " | Navchaetsya na: " << specialty << ", Kurs: " << course << " [" << groupName
<< "]" << endl; Лістинг Б.2 – Клас Student #include "Employee.h" #include <iostream>
5 using namespace std; Employee::Employee() : Person(), company("Nevidomo"),
position("Nevidomo"), salary(0.0) { } Employee::Employee(const string& fullName, int day, int
month, int year, double height, double weight, const string& gender, const string&
phoneNumber, const string& company, const string& position, double salary) :
Person(fullName, day, month, year, height, weight, gender, phoneNumber),
company(company), position(position), salary(salary) { } void Employee::print() const { cout <<
"[ROBITNYK] " << getName() << " (" << getGender() << ")" << " | Data narodzhennya: " <<
getDay() << "." << getMonth() << "." << getYear() << " | Zrist: " << getHeight() << " cm | Vaga: "
<< getWeight() << " kg" << " | Tel: " << getPhone() << " | Misce roboty: " << company << " |
Posada: " << position << " | Zarplata: " << salary << " grn" << endl; Лістинг Б.3 – Клас
Employee
```

Посилання

Це джерела виділених збігів у вашому документі. Кожен збіг позначено темно-зеленим числом, яке відповідає вказаному тут джерелу. Джерела впорядковані за схожістю — чим вищий бал, тим сильніше збіг.

#	Джерело	%
1	dspace.onu.edu.ua	1.1%
2	onu.edu.ua	0.6%
3	lnu.edu.ua	0.6%
4	careers.epam.ua	0.4%
5	ela.kpi.ua	0.4%
6	iir.edu.ua	0.3%
7	dspace.ksaeu.kherson.ua	0.3%
8	conf.ztu.edu.ua	0.3%
9	ela.kpi.ua	0.3%
10	ru.essays.club	0.2%
11	dspace.onu.edu.ua	0.2%
12	mydisser.com	0.2%
13	essuir.sumdu.edu.ua	0.2%
14	medziaga.puslapiai.lt	0.2%



Дякуємо, що перевірили
свій документ за допомогою
Plag!